



Pods in Kubernetes

Understanding Kubernetes for Orchestration

Introduction

When we deploy applications in Kubernetes, we don't work directly with containers. Instead, Kubernetes groups containers inside a unit called a **Pod**. A **Pod** is the smallest deployable unit in Kubernetes. It represents **a single instance of a running process** in the cluster and can contain **one or multiple containers**.

This guide will cover:

- **What are Pods?**
 - **Why does Kubernetes use Pods instead of individual containers?**
 - **Types of Pods**
 - **Creating and managing Pods**
 - **Troubleshooting and FAQs**
-

What are Pods in Kubernetes?

A **Pod** is a group of **one or more containers** that share:

- **Network** – Containers inside a Pod share the same IP address.
- **Storage** – They can share storage volumes.
- **Configuration** – Environment variables and settings can be shared.

A Pod ensures that **related containers** work together as a single unit.

💡 *Example: A web application may need a web server and a logging service in the same Pod.*



Why Do We Need Pods?

Pods provide:

- **Scalability** – We can create multiple replicas of a Pod to handle more traffic.
- **Fault Tolerance** – If a Pod fails, Kubernetes can restart or replace it.
- **Resource Sharing** – Containers inside a Pod share resources efficiently.
- **Simplified Networking** – Containers communicate easily within the same Pod.

💡 *Think of a Pod like a tiffin box that contains multiple food items (containers) packed together!*

Types of Pods

1. Single-Container Pods

- **Most common type** in Kubernetes.
- A **Pod contains only one container**, running an application.
- Simpler to manage, scales easily.

Example: Running an Nginx web server inside a single-container Pod.

2. Multi-Container Pods

- A **Pod contains multiple containers** that work together.
- Containers inside share storage and networking.
- Useful when a **helper container** is needed (e.g., logging or monitoring).

Example: A **web server container** and a **logging agent container** running together in the same Pod.



Creating a Pod in Kubernetes

Pods can be created using **kubectl** commands or **YAML configuration files**.

Method 1: Using **kubectl run** (Command-Line)

To create a Pod named **my-pod** running an **Nginx container**:

```
kubectl run my-pod --image=nginx --restart=Never
```

Check if the Pod is running:

```
kubectl get pods
```

View detailed information about the Pod:

```
kubectl describe pod my-pod
```

Method 2: Using YAML Configuration

Create a file named **pod.yaml**:

```
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
spec:
  containers:
    - name: nginx-container
      image: nginx
      ports:
        - containerPort: 80
```

Apply the configuration:



```
kubectl apply -f pod.yaml
```

Verify the Pod:

```
kubectl get pods
```

💡 *Using YAML is recommended for managing Pods in production environments.*

Accessing a Running Pod

To access a Pod's container:

```
kubectl exec -it my-pod -- /bin/bash
```

To check logs for debugging:

```
kubectl logs my-pod
```

To expose a Pod as a service:

```
kubectl expose pod my-pod --port=80 --type=NodePort
```

Retrieve the external service URL:

```
minikube service my-pod --url
```

Deleting a Pod

To delete a Pod:

```
kubectl delete pod my-pod
```

💡 *Once deleted, Kubernetes will not recreate the Pod unless managed by a Deployment.*



Common Issues and Troubleshooting

Problem	Solution
Pod stuck in Pending state	Check available resources using kubectl describe pod <pod-name>
Pod restarts frequently	Use kubectl logs <pod-name> to check errors
Cannot access Pod from outside	Expose the Pod using a Service (kubectl expose pod)
ImagePullBackOff error	Check if the image name is correct and available in the container registry

Know More (FAQs)

1. What is the difference between a Pod and a Container?

- A **Container** runs a single application.
- A **Pod** is a wrapper that may contain **one or more containers** working together.

2. Can a Pod have multiple containers?

Yes, a Pod can have multiple containers **sharing storage and network**.

3. What happens if a Pod fails?

- If the Pod is part of a **Deployment or ReplicaSet**, Kubernetes automatically replaces it.
- If it's a standalone Pod, it will not restart unless manually recreated.



4. How do I restart a Pod?

Pods cannot be restarted directly. You must delete and recreate them:

```
kubectl delete pod my-pod  
kubectl apply -f pod.yaml
```

5. How do I scale a Pod?

Use a **Deployment** instead of a single Pod and scale it using:

```
kubectl scale deployment my-app --replicas=3
```

Conclusion

Pods are **the foundation of Kubernetes**. They allow us to **run, manage, and scale containerized applications efficiently**.

👉 Try deploying a Pod in your Kubernetes cluster and explore how it works!